House Price Prediction

Document Type: Project Review 1 Report (Exploratory Data Analysis & Data Preprocessing)

Field	Details
Project Title	House Price Prediction
Course / Subject	Machine Learning
Submitted By	*[Student Name(s) and Roll Number(s)]*
Guide / Faculty	*[Faculty Name]*
Institution	*[Institution Name]*
Review	Review 1 of 3
Date	28 October 2026

> **Note on figures:** Numerical values in the tables below (row counts, percentages, statistics) are illustrative placeholders. Replace them with the outputs of the accompanying notebook on your own dataset before submission. The methodology, formulas and code are directly reusable.

---

1. Introduction & Project Overview

1.1 Problem Statement

Property valuation is a high-stakes decision for buyers, sellers, lenders and investors. Manual appraisal is slow, subjective and inconsistent, and market prices depend jointly on many interacting factors such as built-up area, neighbourhood, size of the home, age of the structure, furnishing and proximity to amenities. The problem addressed in this project is to **build a supervised regression model that estimates the market price of a residential property from its structural, locational and contextual attributes.**

1.2 Objective & Scope of Review 1

The overall project objective is an accurate, interpretable House Price Predictor. This report covers **only Review 1**, whose scope is limited to:

1. **Exploratory Data Analysis (EDA):** understanding the target distribution, feature distributions, feature–target relationships and inter-feature correlations.
2. **Data Preprocessing:** missing value handling, outlier detection and removal (IQR method), categorical encoding and feature scaling/standardization.

Model training, hyperparameter tuning and evaluation are deferred to Review 2.

1.3 Dataset Description & Key Features

The dataset consists of historical residential property listings, each described by the following attributes.

Feature	Type	Description
`area_sqft`	Numerical (continuous)	Built-up / carpet area in square feet
`location`	Categorical (nominal)	Locality or city zone of the property
`bedrooms`	Numerical (discrete)	Number of bedrooms
`bathrooms`	Numerical (discrete)	Number of bathrooms
`property_age`	Numerical (continuous)	Age of the property in years
`furnishing_status`	Categorical (ordinal/nominal)	Unfurnished, Semi-furnished, Furnished
`nearby_facilities`	Numerical / Categorical	Count or indicator of nearby schools, hospitals, transit, markets
`price`	Numerical (continuous)	**Target variable**: market price of the house

Dataset snapshot (illustrative):

Property	Value
Total records (raw)	1,000
Total features (excluding target)	7

Property	Value
Numerical features	5
Categorical features	2
Target variable	`price` (continuous, regression task)

Initial inspection was performed with the following code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("house_data.csv")

print(df.shape)
print(df.head())
df.info()
print(df.describe(include="all"))
```

---

2. Exploratory Data Analysis (EDA)

EDA was carried out to (i) understand the statistical character of the data, (ii) identify data quality issues, and (iii) guide the preprocessing and modelling decisions that follow.

2.1 Data Distribution & Skewness Analysis (Target Variable: Price)

```
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
sns.histplot(df["price"], kde=True, bins=40)
plt.title("Distribution of House Price")

plt.subplot(1, 2, 2)
sns.histplot(np.log1p(df["price"]), kde=True, bins=40)
plt.title("Distribution of log(1 + Price)")
plt.show()

print("Skewness (raw):", df["price"].skew())
print("Skewness (log):", np.log1p(df["price"]).skew())
```

Graph description. The histogram of `price` with a kernel density estimate (KDE) overlay shows a **right-skewed (positively skewed) distribution**: the bulk of properties are concentrated in the low-to-mid price range, while a long tail extends toward expensive properties. The mean exceeds the median, which is the classical signature of positive skew.

Statistic	Raw Price	log(1 + Price)
Skewness	≈ 1.8 (illustrative)	≈ 0.2 (illustrative)
Mean vs. Median	Mean > Median	Approximately equal

Interpretation. A heavily skewed target violates the approximate-normality assumption of residuals in linear models and lets a few expensive properties dominate squared-error loss. A logarithmic transformation of the target is therefore flagged as a candidate step for Review 2, while tree-based models are largely insensitive to it.

2.2 Univariate Feature Analysis

```
num_cols = ["area_sqft", "bedrooms", "bathrooms", "property_age"]
df[num_cols].hist(bins=30, figsize=(12, 8))
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
sns.countplot(x="furnishing_status", data=df, ax=ax[0])
sns.countplot(y="location", data=df, ax=ax[1],
              order=df["location"].value_counts().index)
plt.show()
```

Feature	Observed Behaviour
`area_sqft`	Right-skewed; most homes lie in a moderate range, with a few very large properties forming outliers.
`bedrooms` / `bathrooms`	Discrete, concentrated at 2–3 bedrooms and 1–2 bathrooms.
`property_age`	Broad spread from new construction to several decades old.

Feature	Observed Behaviour
`furnishing_status`	Reasonably balanced across the three categories, with semi-furnished typically the largest group.
`location`	Uneven class frequencies; a few localities hold a large share of listings, others are sparse.

2.3 Bivariate Feature Analysis

(a) Area vs. Price

```
sns.scatterplot(x="area_sqft", y="price", data=df, alpha=0.6)
sns.regplot(x="area_sqft", y="price", data=df, scatter=False, color="red")
plt.title("Area vs. Price")
plt.show()
```

The scatter plot reveals a **strong positive, approximately linear relationship** between area and price: larger houses command higher prices. The spread (variance) of price increases with area, indicating **heteroscedasticity**, and a handful of points lie far above the trend line, which are candidates for outlier treatment.

(b) Furnishing Status vs. Price

```
sns.boxplot(x="furnishing_status", y="price", data=df,
            order=["Unfurnished", "Semi-furnished", "Furnished"])
plt.title("Price by Furnishing Status")
plt.show()
```

The box plots show a **monotonic upward shift in median price** from unfurnished to semi-furnished to furnished homes. Interquartile ranges overlap, so furnishing is an informative but not a standalone determinant of price. Individual points beyond the whiskers confirm the presence of high-value outliers in each category.

(c) Location Impact

```
order = df.groupby("location")
["price"].median().sort_values(ascending=False).index
plt.figure(figsize=(10, 5))
sns.boxplot(x="location", y="price", data=df, order=order)
plt.xticks(rotation=45)
plt.title("Price by Location")
plt.show()
```

Location shows **the largest between-group variation** in median price among the categorical features. Premium localities have both higher medians and wider spreads, while peripheral localities cluster at lower price levels. This confirms that location is a first-order predictor and must be encoded carefully.

(d) Bedrooms, Bathrooms and Property Age vs. Price

- Price rises with the number of bedrooms and bathrooms, with diminishing marginal increments at higher counts.
- Property age shows a **mild negative association** with price, reflecting depreciation, although well-located older homes can retain high value.

2.4 Correlation Matrix & Heatmap Insights

```
corr = df.select_dtypes(include="number").corr()
plt.figure(figsize=(8, 6))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Correlation Heatmap")
plt.show()

print(corr["price"].sort_values(ascending=False))
```

The Pearson correlation coefficient between two variables X and Y is defined as:

```
r = Cov(X, Y) / (σ_X · σ_Y)        where  −1 ⩽ r ⩽ +1
```

Feature Pair	Correlation (illustrative)	Insight
`area_sqft` – `price`	+0.75	Strongest linear predictor of price.
`bathrooms` – `price`	+0.50	Moderate positive relationship.
`bedrooms` – `price`	+0.40	Moderate positive relationship.

Feature Pair	Correlation (illustrative)	Insight
`property_age` – `price`	−0.25	Weak negative relationship.
`area_sqft` – `bedrooms`	+0.55	Moderate **multicollinearity** risk.
`bedrooms` – `bathrooms`	+0.45	Related size indicators.

Insights. Area is the dominant numerical driver of price. Moderate correlation among size-related features (area, bedrooms, bathrooms) suggests mild multicollinearity, which may inflate coefficient variance in linear regression; this will be monitored using the Variance Inflation Factor (VIF) in Review 2. No pair exceeds |r| = 0.85, so no feature is dropped solely for redundancy at this stage.

2.5 Key Insights & Patterns Discovered

1. The target `price` is **right-skewed**, motivating a log transformation or robust, tree-based models.
2. **Area** has the strongest linear relationship with price, followed by bathrooms and bedrooms.
3. **Location** is the most influential categorical variable; price varies substantially across localities.
4. **Furnishing status** shifts median price upward in a consistent order.
5. **Property age** has a mild negative effect on price.
6. Several numerical features contain **extreme values** (visible in box plots and scatter plots) requiring systematic outlier treatment.
7. Size-related features are moderately inter-correlated, indicating possible multicollinearity.

---

3. Data Preprocessing & Cleaning

The preprocessing pipeline is executed in the following order, chosen to prevent data leakage and to ensure each step operates on appropriately prepared data:

```
Raw Data → Missing Value Imputation → Outlier Removal (IQR)
         → Train/Test Split → One-Hot Encoding → Standardization → Model-ready Data
```

3.1 Missing Value Analysis & Imputation Strategy

```
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(2)
print(pd.DataFrame({"Missing": missing, "Percent": missing_pct})
      .query("Missing > 0"))

sns.heatmap(df.isnull(), cbar=False)
plt.title("Missing Value Map")
plt.show()
```

Feature	Type	Missing % (illustrative)	Strategy
`area_sqft`	Numerical	2.0%	Median imputation
`bathrooms`	Numerical	1.5%	Median imputation
`property_age`	Numerical	3.0%	Median imputation
`furnishing_status`	Categorical	1.0%	Mode imputation
`location`	Categorical	0.5%	Mode imputation

Rationale.

- **Median for numerical features:** the median is robust to outliers and skewed distributions, both present in this dataset. The mean would be pulled upward by extreme values (e.g., very large areas).
- **Mode for categorical features:** the most frequent category is the only central-tendency measure defined for nominal data and introduces minimal distortion when missingness is low.
- Rows with a missing **target** value are dropped, because imputing the label would introduce artificial supervision.
- Because the fraction of missing data is small (< 5% per column), imputation preserves sample size without materially biasing distributions.

```
df = df.dropna(subset=["price"])

num_cols = ["area_sqft", "bedrooms", "bathrooms", "property_age"]
cat_cols = ["location", "furnishing_status"]

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())
for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

assert df.isnull().sum().sum() == 0
```

> *Best-practice note:* in the final modelling pipeline, medians and modes should be computed on the training split only (e.g., via `sklearn.impute.SimpleImputer` inside a `Pipeline`) to avoid leakage.

3.2 Outlier Detection & Removal (Interquartile Range Method)

Concept. The IQR method is a non-parametric technique that identifies outliers using quartiles, making it robust to non-normal and skewed data (unlike the Z-score method, which assumes approximate normality).

Let Q1 be the 25th percentile and Q3 the 75th percentile. The interquartile range is:

```
IQR = Q3 − Q1
```

The outlier fences (Tukey's fences) are:

```
Lower Bound = Q1 − 1.5 × IQR
Upper Bound = Q3 + 1.5 × IQR
```

Any observation x satisfying `x < Lower Bound` or `x > Upper Bound` is flagged as an outlier.

Implementation.

```
def iqr_bounds(series, k=1.5):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    return q1 - k * iqr, q3 + k * iqr

outlier_cols = ["area_sqft", "price", "property_age"]
n_before = len(df)

for col in outlier_cols:
    lo, hi = iqr_bounds(df[col])
    print(f"{col}: lower={lo:.2f}, upper={hi:.2f}")
    df = df[(df[col] >= lo) & (df[col] <= hi)]

print(f"Rows removed: {n_before - len(df)} ({(n_before - len(df)) /
n_before:.1%})")
```

Illustrative outcome.

Feature	Q1	Q3	IQR	Lower Bound	Upper Bound	Outliers Removed
`area_sqft`	1,200	2,000	800	0	3,200	35
`price`	45 L	95 L	50 L	−30 L	170 L	40
`property_age`	3	15	12	−15	33	10

(L = lakh; replace with your own units and computed values.)

Box plots before and after treatment confirm that points beyond the whiskers have been eliminated and that the distributions of `area_sqft` and `price` are more compact and symmetric.

Justification and caveats.

- Extreme values distort mean-based statistics, inflate variance and disproportionately influence squared-error-based models such as linear regression.
- The multiplier k = 1.5 is the standard Tukey convention; k = 3 may be used for “far-out” outliers if the data loss proves excessive.
- A negative lower bound for naturally non-negative variables is simply clipped at the feasible minimum.
- Removal is limited to values that are statistically extreme *and* not structurally meaningful; the proportion removed is kept small (target below roughly 5–8%) so that genuine luxury properties are not systematically excluded.

3.3 Categorical Feature Encoding (One-Hot Encoding)

Machine learning algorithms require numeric inputs. Label encoding of nominal variables (e.g., Location A = 0, B = 1, C = 2) would impose a false ordinal relationship. **One-Hot Encoding (OHE)** avoids this by creating one binary indicator column per category.

Example (`furnishing_status`):

furnishing_status	Furnished	Semi-furnished	Unfurnished
Furnished	1	0	0
Semi-furnished	0	1	0
Unfurnished	0	0	1

```
df_encoded = pd.get_dummies(df, columns=["location", "furnishing_status"],
                            drop_first=True, dtype=int)
print(df_encoded.shape)
```

Design choices.

- `drop_first=True` removes one dummy per feature to avoid the **dummy variable trap** (perfect multicollinearity), which would make the design matrix of a linear model singular.
- If the `location` feature has high cardinality, rare categories can be grouped into an "Other" bucket before encoding to limit dimensionality.
- In the final pipeline, `OneHotEncoder(handle_unknown="ignore")` is used within a `ColumnTransformer` so unseen categories at inference time do not raise errors.

3.4 Feature Scaling & Standardization

Rationale. Features in the dataset operate on very different scales (e.g., `area_sqft` in the thousands versus `bathrooms` in single digits). Scaling is necessary because:

1. **Distance-based and gradient-based algorithms** (KNN, SVR, neural networks, regularized linear models) are sensitive to feature magnitude; large-scale features would otherwise dominate.
2. **Gradient descent converges faster** on standardized features, since the loss surface is better conditioned.

3. **Regularization (Ridge/Lasso)** penalizes coefficients uniformly, which is meaningful only when features share a comparable scale.
4. Coefficient magnitudes become **directly comparable** as a measure of relative importance.

Standardization (Z-score transformation) via `StandardScaler`:

```
z = (x − μ) / σ

μ = (1/n) Σ xᵢ                    (feature mean)
σ = sqrt( (1/n) Σ (xᵢ − μ)² )     (feature standard deviation)
```

After transformation, each scaled feature has **mean 0 and standard deviation 1**. For reference, Min-Max normalization, `x' = (x − x_min) / (x_max − x_min)`, maps values to [0, 1] but is more sensitive to residual outliers, which is why standardization is selected here.

Implementation (with leakage prevention):

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df_encoded.drop(columns=["price"])
y = df_encoded["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

num_cols = ["area_sqft", "bedrooms", "bathrooms", "property_age"]
scaler = StandardScaler()
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])  # fit on train
only
X_test[num_cols]  = scaler.transform(X_test[num_cols])       # apply train
statistics
```

Important points.

- The scaler is **fitted on the training set only** and then applied to the test set, preventing information from the test distribution from leaking into training.
- Binary one-hot columns are **not** scaled, as they are already on a 0/1 scale and scaling would reduce interpretability.

- Tree-based models (Decision Tree, Random Forest, XGBoost) are scale-invariant, but scaled data is retained so that all candidate models can be compared on an identical feature set.

Verification:

```
print(X_train[num_cols].mean().round(3))   # ≈ 0
print(X_train[num_cols].std().round(3))    # ≈ 1
```

---

4. Summary & Next Steps for Review 2

4.1 Dataset Readiness Summary

Preprocessing Stage	Technique Applied	Status
Data understanding	Descriptive statistics, histograms, box plots, scatter plots, correlation heatmap	Completed
Missing values	Median (numerical), Mode (categorical), row-drop for missing target	Completed
Outliers	IQR method (Q1 − 1.5·IQR, Q3 + 1.5·IQR)	Completed
Categorical encoding	One-Hot Encoding with `drop_first=True`	Completed
Feature scaling	StandardScaler (Z-score), fitted on training data	Completed
Data split	80% training / 20% testing, `random_state=42`	Completed

The dataset is now **free of missing values, free of extreme outliers, fully numeric and standardized**, and is therefore ready for model training. The principal findings that will shape modelling are the right-skewed target, the dominance of area and location as predictors, and mild multicollinearity among size-related features.

4.2 Proposed Regression Models for Review 2

Model	Rationale	Notes
Linear Regression	Interpretable baseline; captures the strong linear area–price trend.	Check residual assumptions and VIF.

Model	Rationale	Notes
Ridge / Lasso Regression	Regularization to handle multicollinearity and feature selection.	Tune α via cross-validation.
Decision Tree Regressor	Captures non-linear interactions; easy to visualize.	Prone to overfitting; tune depth.
Random Forest Regressor	Ensemble of trees; reduces variance and is robust to noise.	Provides feature importances.
XGBoost Regressor	Gradient boosting; typically state-of-the-art on tabular data.	Tune learning rate, depth, estimators.

Planned evaluation framework:

- **Metrics:** MAE, MSE, RMSE and the coefficient of determination (R^2).
- **Validation:** k-fold cross-validation (k = 5) and hyperparameter tuning with `GridSearchCV` / `RandomizedSearchCV`.
- **Target handling:** compare training on raw price versus log-transformed price.
- **Interpretability:** feature importance and SHAP analysis for the best-performing model.

Submitted to - Nirbhay Gupta